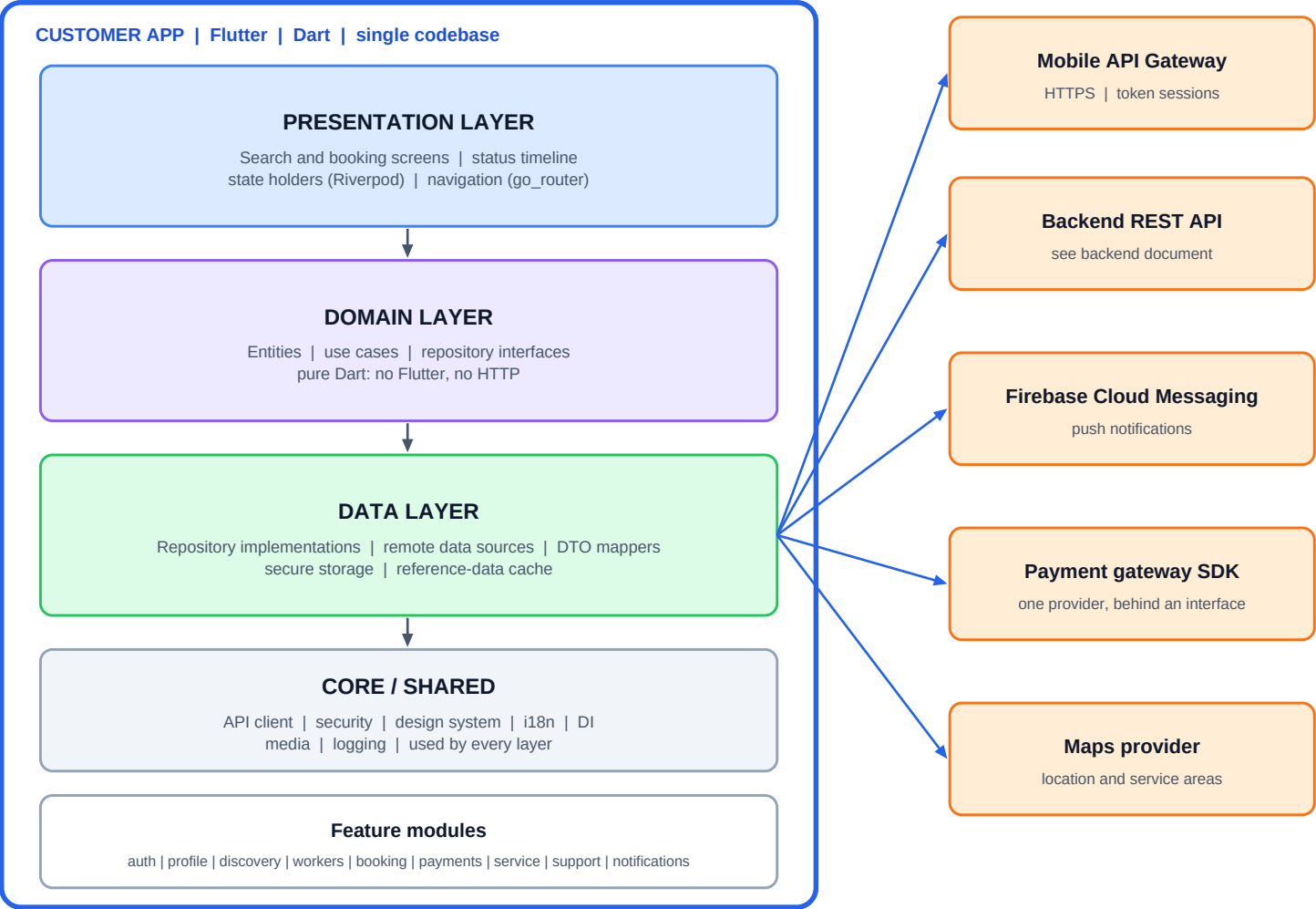


SEVANEST — CUSTOMER APP FRONTEND ARCHITECTURE

Flutter Architecture for the Customer Mobile Application (Android & iOS)



Calls flow downward. The Data layer implements Domain interfaces; the Domain layer never depends on Flutter or HTTP.

One Flutter codebase serves Android and iOS. Every backend call goes through the Mobile API Gateway over HTTPS with token-based sessions; only maps, push and the payment SDK are reached directly on the device.

■ Presentation / UI
■ Domain / app logic
■ Data / local storage
■ Device & external services
■ Security-sensitive
■ Core / shared / infrastructure

THE ARCHITECTURE IN SIMPLE TERMS

- Presentation.** Short search and booking screens with a clear status timeline. No API calls and no business rules here.
- Domain.** The rules: entities and use cases such as SearchWorkers, RequestInterview, ConfirmBooking.
- Data.** Talks to the backend, secure storage and reference-data cache; turns API errors into domain failures.
- Core / Shared.** API client, security, design system, localization and logging, shared with the Worker App.
- Backend.** The server is the source of truth for booking, payment and attendance status.

DECISION STATUS

Decision	Status
Flutter + Dart	Specified by the PRD
Firebase FCM	Specified by the PRD
Payment provider	One of the PRD options
Riverpod, go_router, dio	Proposed
Telugu / Hindi UI	Phase 2 (not V1)
Numeric perf budgets	Open item

ARCHITECTURAL DRIVERS

Goal	How the architecture addresses it
Find a worker in a few steps	Short discovery flow, saved addresses, simple filters and clear worker cards
Trust at the point of choice	Verification badges and ratings on every worker card; a badge appears only when the server says approved
Reliable booking and payment	Gateway SDK behind an interface; server status is the truth; idempotency keys prevent duplicates
Transparent status	Booking timeline driven by server status and push updates
Recurring and short-term needs	Engagement type (full-time, part-time, daily, hourly, live-in) is data, not separate flows
Local-language readiness	All text externalized; layouts tolerate longer text
Privacy	Customers see worker cards and badges only; no full identity documents are ever shown
Single codebase for Android and iOS	Flutter with platform-specific code isolated behind interfaces
Maintainable by a small team	Layered, feature-first modules and a core library shared with the Worker App

PROPOSED Items marked Proposed are architectural choices or assumptions not fixed by the PRD. They are confirmed during the architecture phase.

TECHNOLOGY STACK

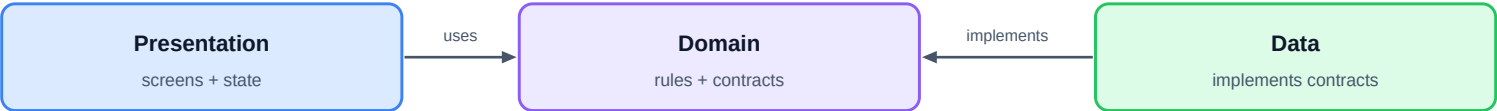
Area	Choice	Notes
Framework / language	Flutter, Dart	PRD recommendation
State management (Proposed)	Riverpod (alternative: Bloc)	Same as Worker App
Navigation (Proposed)	go_router	Declarative routes, login redirect, notification deep links
HTTP client (Proposed)	dio with interceptors	Token handling, retry, error mapping
Secure storage (Proposed)	flutter_secure_storage	Session tokens
Reference-data cache (Proposed)	Lightweight local storage	Categories, areas, languages
Payments	Razorpay / Cashfree / PayU SDK	PRD options; one provider behind PaymentService
Push notifications	Firebase Cloud Messaging	PRD recommendation
Maps / location	Google Maps or equivalent	Location and area selection for search
Localization (Proposed)	Flutter intl with ARB files	Readiness for Telugu, Hindi in Phase 2
Crash reporting (Proposed)	Firebase Crashlytics	Confirm tool choice
Testing	flutter_test, mocktail, integration_test	Plus payment success and failure tests
CI/CD	Pipeline for build, test, signing, store upload	Environments defined by DevOps engineer

Why the stack matches the Worker App. Flutter is specified by the PRD. The other entries are proposed and match the Worker App so that the design system, API client, storage and notification code can be shared.

LAYER RESPONSIBILITIES

Layer	Responsibility	Rules
Presentation	Search and booking screens, filters, status timeline, navigation events and state holders	No API calls or business rules
Domain	Entities, use cases (for example SearchWorkers, RequestInterview, ConfirmBooking, RequestReplacement) and repository interfaces	Pure Dart
Data	Repository implementations, remote data sources, DTOs, mappers, secure storage, reference-data cache	Implements domain interfaces; maps API errors to domain failures
Core / Shared	API client, security, design system, localization, error and logging utilities	No feature-specific logic

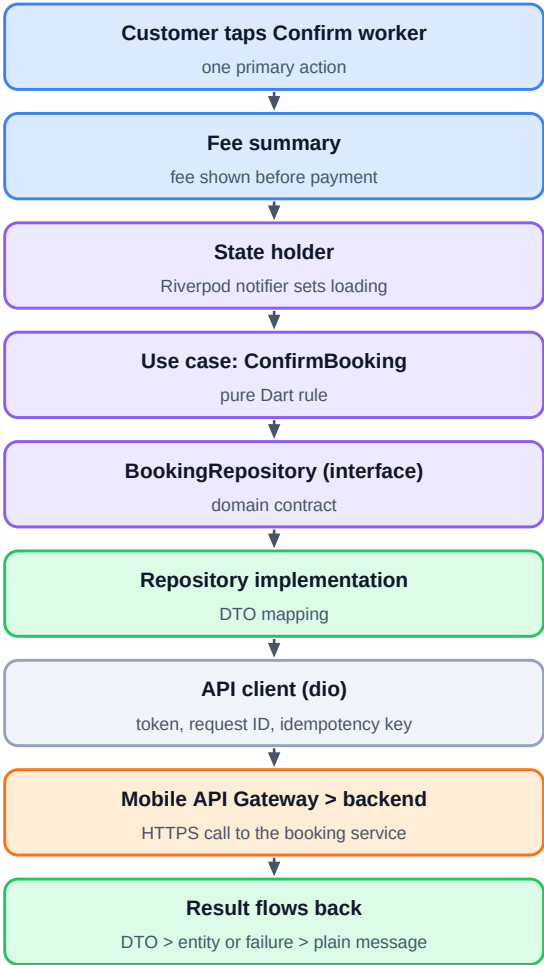
DEPENDENCY RULE



COMMUNICATION WITH THE BACKEND

- All calls go through the Mobile API Gateway over HTTPS with token-based sessions.
- Each feature talks to its matching backend service area: Authentication, User/Profile, Service/Category and Availability, Matching/Search, Booking, Payment, Notification and Support/Complaint.
- Only on-device services (maps, FCM, payment SDK) are accessed directly.

EXAMPLE: CONFIRM A BOOKING



Error handling. Failures return as domain errors (network, unauthorized, validation, not found, conflict, server, unknown) and are shown as one short sentence with one next step.

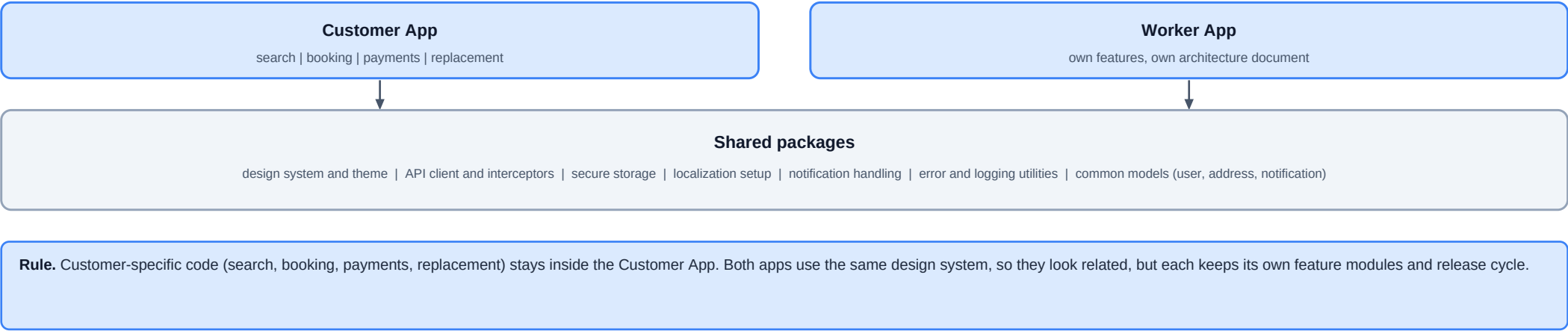
FEATURE-FIRST FOLDER LAYOUT (PROPOSED)

```
customer_app/  
  lib/  
    main_dev.dart, main_stage.dart, main_prod.dart (flavors)  
  app/      app widget, router, theme wiring, bootstrap  
  core/     network, security, storage, error, logging  
  shared/   design system widgets, status labels, l10n  
  features/  
    auth/      OTP login, session  
    profile/   profile, saved addresses, language  
    discovery/ category, engagement type, location, timings, budget  
    workers/   worker list, filters, profile, shortlist  
    booking/   interview/trial, confirmation, timeline  
    payments/  fee payment, receipts, history, refunds  
    service/   current service, attendance, history, rating, replacement  
    support/   requests and complaints  
    notifications/ push handling, inbox, deep links  
    <feature>/{presentation, domain, data}  
  test/    integration_test/
```

FEATURE MODULES AND MAIN SCREENS

Feature module	Main screens
auth	Splash, Login, OTP entry
profile	Profile (name, email, preferred language), Saved addresses
discovery	Service category, Engagement type, Location, timings, start date, budget
workers	Worker list with filters, Worker profile (photo, experience, skills, language, area, availability, badge, ratings), Shortlist
booking	Interview/trial request, Date and time selection, Worker confirmation, Booking status timeline
payments	Pay booking fee, Receipt, Transaction history, Refund status
service	Current service, Attendance / status, Service history, Rating, Replacement request
support	Create request / complaint, Request status
notifications	Notification inbox, push deep links

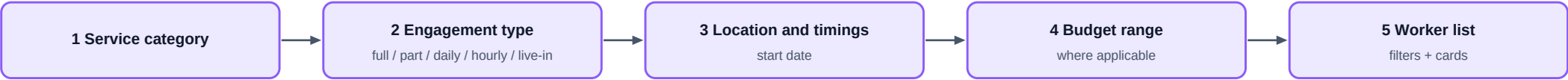
CODE SHARING WITH THE WORKER APP (PROPOSED)



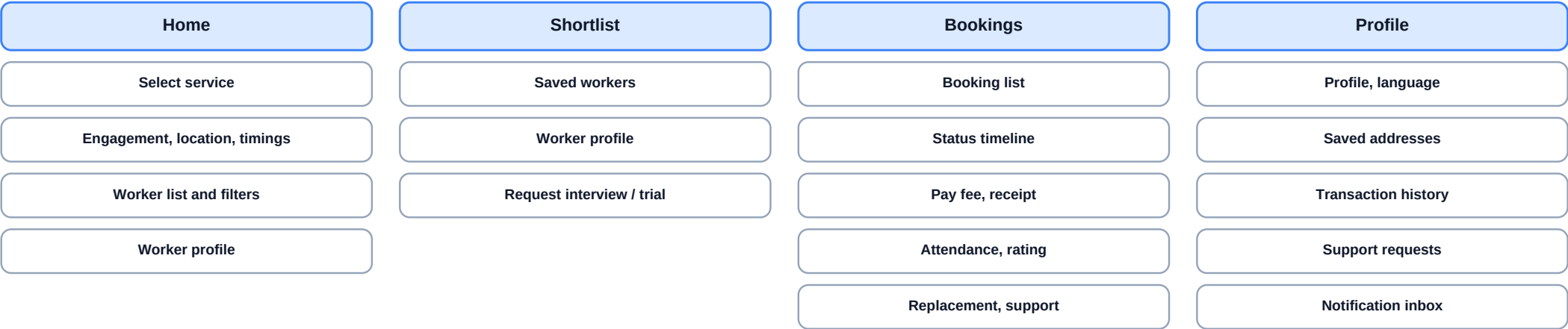
APP FLOW



SEARCH STEPS - STATE KEPT UNTIL BOOKING OR RESET

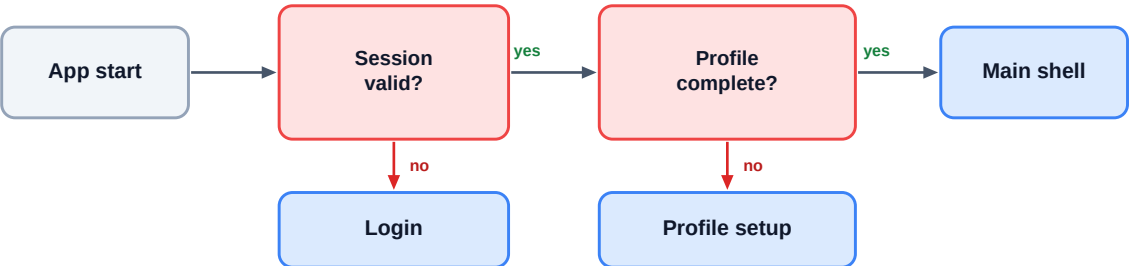


MAIN SHELL TABS



Push notifications deep-link to bookings, payments and support requests. Short flows and plain status labels throughout.

REDIRECT RULES

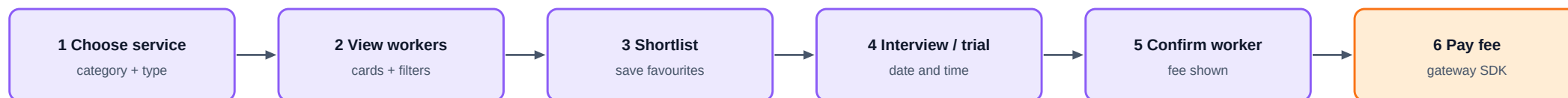


Presentation / UI Domain / app logic Data / local storage Device & external services Security-sensitive Core / shared / infrastructure

NAVIGATION RULES

- Router is declarative; routes are named and typed.
- Not logged in goes to Login; logged in without a complete profile goes to Profile setup; otherwise to the main shell.
- Main shell has four tabs (Home, Shortlist, Bookings, Profile). Support is reached from Bookings and Profile.
- A visible back option and a plain home action exist on every screen.
- Push notifications carry a route and identifier so the app opens the right booking, payment or support request.

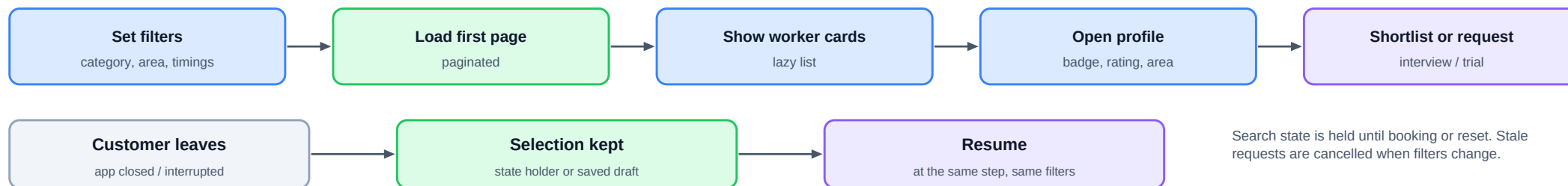
HIRING JOURNEY - A FEW STEPS, ONE PURPOSE EACH, ONE CLEAR PRIMARY ACTION



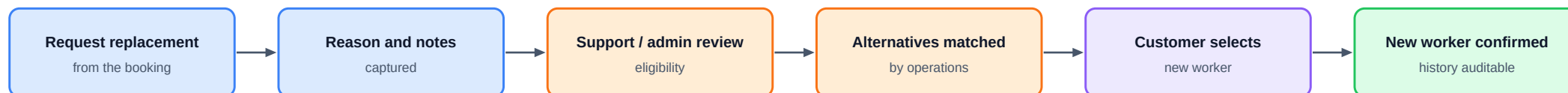
Progress is visible on every step. Selections are kept in a feature state holder until the booking is made or the customer resets.



LIFECYCLE OF EVERY SEARCH



REPLACEMENT JOURNEY



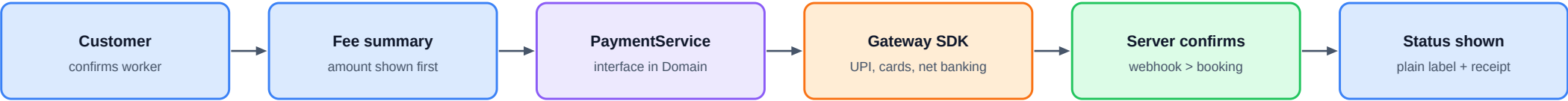
The app only collects the reason and shows the status; eligibility, matching and the audit history stay on the server. The previous engagement remains visible in service history.

BOOKING RULES IN THE APP

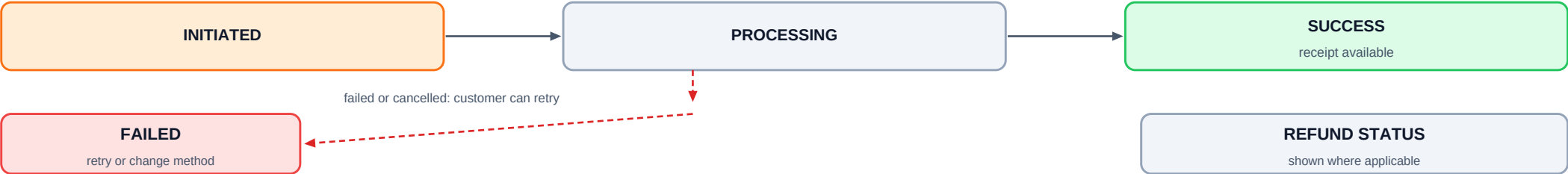
- A worker can be booked only for the engagement type the worker offers; unavailable options are not shown.
- Interview or trial is requested first; payment follows confirmation of the worker.
- Cancelled, replaced and completed bookings stay in history.

Why this design. The PRD asks that customers find a suitable worker in a few steps. Short steps, saved selections and one dominant action per screen keep hiring quick for first-time and repeat customers.

PAYMENT PIPELINE



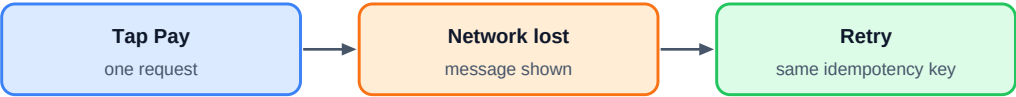
PER-PAYMENT STATUS



The app shows the payment status returned by the server, not the SDK callback alone. If the app is closed during payment, the status is fetched again on next open.

RETRY BEHAVIOUR

PROPOSED



A retry reuses the same idempotency key, so repeated taps or a lost response produce one payment and one booking update.

PRIVACY AND SECURITY RULES

- Card and UPI details are entered in the gateway SDK; they never touch app code, storage or logs.
- The app never holds gateway secrets; payment orders are created by the backend.
- Receipts and transaction history are fetched from the server and show masked references only.
- The payment provider sits behind a PaymentService interface so it can be changed without touching screens.

RECEIPTS AND HISTORY

- Receipt per successful payment, available from the booking and from Transaction history.
- Refund status is read-only and comes from the server.

Same rule on both sides. Aligned with the backend: the server owns payment and booking state, creates payment orders, verifies gateway results and keeps an audit record. The app only presents what the server reports.

DESIGN RULES (VALUES SUBJECT TO DESIGN REVIEW)

Rule	Standard (Proposed)
Clear steps	Short flows; one purpose per screen; details behind a clearly labelled option
Touch targets	At least the platform minimum (about 48 dp); full-width primary button
Icons with labels	Every icon has a text label; supports recognition and later translation
One primary action	Each screen has one dominant action
Readable type	Respects system font scaling; layouts tolerate longer translations
Contrast and colour	Status never relies on colour alone: text and icon accompany colour
Trust cues	Verification badge and rating summary visible on every worker card
Forgiving input	Saved selections, easy edit, confirmation before irreversible actions such as booking, payment or cancel
Shared look	Same design system as the Worker App, with standard sizing tokens

STATUS LABEL MAPPING (PROPOSED)

Area	Backend status (examples)	Customer label
Verification	Approved	Verified badge
Verification	Not approved / in review	No badge
Booking	Interview requested	Interview requested
Booking	Interview / trial scheduled	Interview set
Booking	Confirmed	Worker confirmed
Booking	Active	Service active
Booking	Completed / Cancelled	Completed / Cancelled
Booking	Replacement requested	Replacement in progress
Payment	Success	Paid
Payment	Failed	Payment failed
Payment	Refund pending	Refund status

One mapping class converts a backend status into a short label, an icon and a next action. Unknown statuses fall back to a generic label instead of failing.

REUSABLE COMPONENTS (STATELESS, DRIVEN BY DATA)

Primary button

Worker card

Verification badge

Rating stars

Filter chip

Status timeline

Payment summary

Address picker

Empty and error views

HOW LABELS LOOK ON SCREEN

Verified

Interview set

Pay fee

Payment failed

Paid

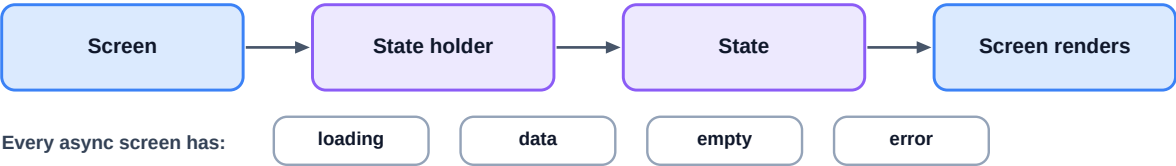
Replacement in progress

Why stateless. Stateless components keep the UI testable and consistent with the Worker App design system. They support the PRD aim of simple discovery and booking, with plain status labels and local-language readiness.

STATE BY FEATURE (RIVERPOD, PROPOSED)

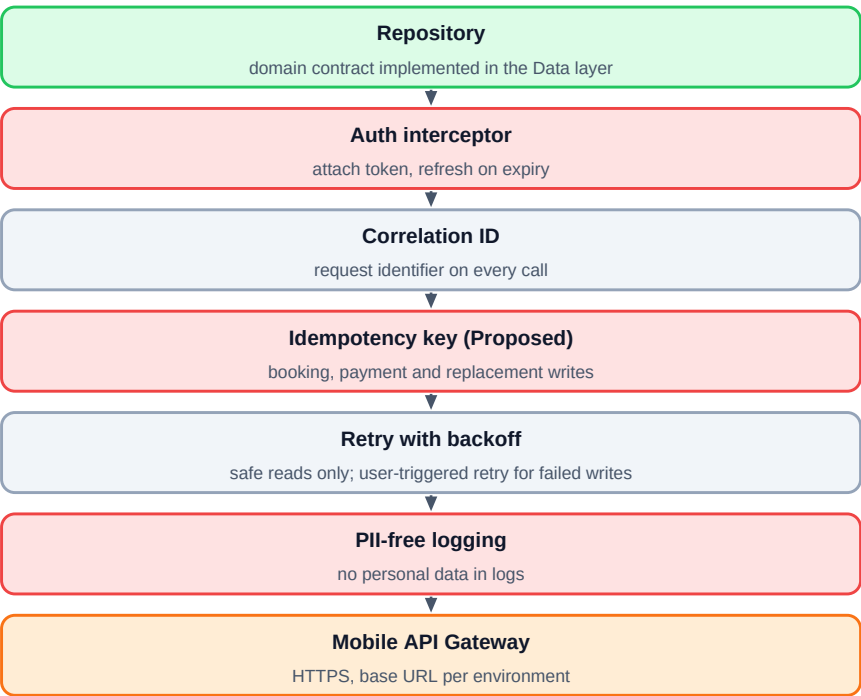
Feature	State held	Lifetime
Session (global)	Authentication status, user identity, language	App lifetime
Discovery	Selected category, engagement type, location, timings, budget	Until booking or reset
Workers	Result list, filters, shortlist, selected worker	Feature
Booking	Interview/trial request, selected slot, booking status	Feature
Payments	Payment progress, receipts, transaction history	Feature
Service	Current service, attendance, replacement request	Feature
Support	Request list and draft	Feature
Notifications	Unread count, inbox	App lifetime

HOW A SCREEN RENDERS

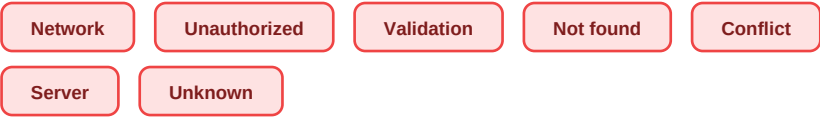


- One state holder per screen or flow, scoped to the feature and disposed with it.
- Immutable state; screens render from state.
- The server is the source of truth for booking, payment and attendance status.

API CLIENT - EVERY CALL PASSES THE SAME PIPELINE



FAILURES MAP TO DOMAIN ERRORS



Repository interfaces live in the Domain layer; implementations live in the Data layer. API responses are mapped from DTOs to domain entities, so screens never see raw API shapes.

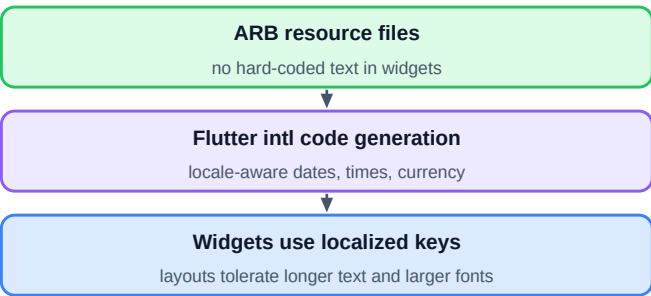
Why idempotency. Idempotency keys make repeated taps safe: a customer who taps Pay or Confirm twice on a weak network produces one payment and one booking, matching the backend's idempotency handling.

WEAK NETWORK BEHAVIOUR (PROPOSED)

Data / action	Behaviour
Worker search and lists	Show last loaded data with a "last updated" indicator; refresh when online
Booking and payment actions	Require network; clear message and retry; never silently dropped
Reference data (categories, areas, languages)	Cached locally and refreshed on launch
Selections in progress	Kept in memory and restored if the app resumes
Session token	Secure storage until logout or expiry

Scope. V1 requires graceful failure on weak networks; a full offline mode is not required.

LOCALIZATION READINESS



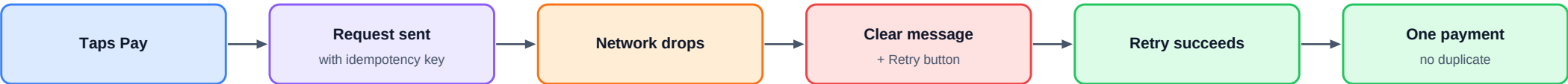
- Preferred language is captured at profile setup and changeable from Profile. The choice is also sent to the backend for notification language.
- V1 ships with languages approved by the business; Telugu, Hindi and others are part of the Phase 2 multi-language UI plan.
- Worker names, areas and skills are shown as provided by the server.

PERFORMANCE APPROACH

Area	Approach
Launch and common screens	Lazy-load features; defer non-essential initialization; stay responsive on normal mobile networks
Lists	Pagination and lazy builders for worker results
Images	Cached thumbnails sized for worker cards
Network	Cancel stale search requests; avoid chatty calls; retry with backoff
Device range (Proposed)	Test on mid and low-end devices and small screens
Memory	Dispose controllers and state holders with their features

Open item. The PRD sets qualitative targets only. Numeric performance budgets are to be agreed before performance testing.

WHAT A CUSTOMER SEES ON A WEAK NETWORK



Principle. Writes are never dropped silently: the customer always sees either success or a plain message with a next step. Reads fall back to the last loaded data with a "last updated" indicator.

SECURITY ARCHITECTURE

Concern	Approach
Transport	HTTPS for all traffic; certificate validation enforced; certificate pinning to be evaluated (Proposed)
Authentication	Mobile OTP then token-based session; tokens only in secure storage
OTP abuse	App respects resend timers and attempt limits and shows backend rate-limit messages
Worker data	Worker cards and verification badges only; full identity documents are never shown or fetched
Payments	Handled by the gateway SDK; card and UPI details never touch app code or storage
Local data	No sensitive personal data in plain storage, logs, crash reports or analytics events
Secrets	No API secrets in the app bundle; environment configuration injected at build time
Account state	Suspended customers are signed out or restricted and shown a clear message
Release hardening	Code obfuscation and debug-symbol upload for release builds (Proposed)
Privacy and consent	Terms, privacy policy and consent flows shown per legally reviewed content before production release; retention rules are an open item

WHERE DATA LIVES ON THE DEVICE



INTEGRATIONS

Integration	Frontend approach
Push (FCM)	Token registration after login and refresh on change; foreground, background and cold-start handling through the notifications module, with deep links to bookings, payments and support
Payments	One provider SDK behind a PaymentService interface; result reflected from server status
Maps / location	Location and area selection for search; manual area entry if permission is denied
SMS OTP	Delivered by the backend; automatic OTP reading is optional (Proposed)

OBSERVABILITY AND ERROR HANDLING

- Crash reporting and non-fatal error capture in production, with release symbols.
- Structured logging without personal data.
- Analytics events (Proposed): search started, worker viewed, shortlist added, interview requested, booking confirmed, payment success / failure, replacement requested, support request.
- Error messages are short, plain-language and give one next step; technical detail goes to logs only.

TESTING STRATEGY

Level	Scope	Tooling
Unit	Use cases, repositories, mappers, validators, status mapping, state holders	flutter_test, mocktail
Widget	Design system components and key screens in all states and font scales	flutter_test
Integration	Login, search, interview request, booking with payment success and failure, replacement, support	integration_test
Device and OS	Agreed Android and iOS device matrix	Device farm or physical devices
Usability	Trial with representative customers to validate a few-step hiring flow	Observation and feedback sessions
Non-functional	Performance, security and accessibility basics	Profiling and static analysis
UAT	Critical workflows per acceptance criteria on agreed devices	Manual with business team

ENVIRONMENTS AND FLAVORS

Flavor	Backend	Purpose
dev	Development	Daily development
stage	Staging	QA and UAT
prod	Production	Store release

Environment configuration is injected at build time; no secrets live in the app bundle.

CI/CD AND RELEASE



Semantic versioning with build numbers and a changelog. A minimum-supported-version check lets the backend prompt for updates (Proposed). Production monitoring and crash alerts are active at release.

ARCHITECTURE SUMMARY

Item	Summary
Framework	Flutter / Dart, one codebase for Android and iOS (PRD)
Architecture	Layered clean architecture, feature-first modules, core shared with Worker App
State / navigation	Riverpod go_router with login and profile redirects (Proposed)
Network	dio via Mobile API Gateway; HTTPS + token-based sessions
Payments	Gateway SDK behind PaymentService; server status is the truth
UI standard	Few-step flows, plain status labels, verification badges on every worker card
Localization	ARB / intl readiness; Telugu and Hindi UI in Phase 2

OPEN OR PROPOSED ITEMS

- Numeric performance budgets
- Payment provider selection
- Certificate pinning evaluation
- Automatic OTP reading (optional)
- V1 language list approved by the business
- Data retention rules
- Crash / analytics tool choice
- Agreed device and OS matrix

The SevaNest Customer App is designed as a simple, layered and feature-first Flutter application: customers can find verified workers, book and pay in a few steps and track the engagement with plain status labels, while the backend remains the source of truth.